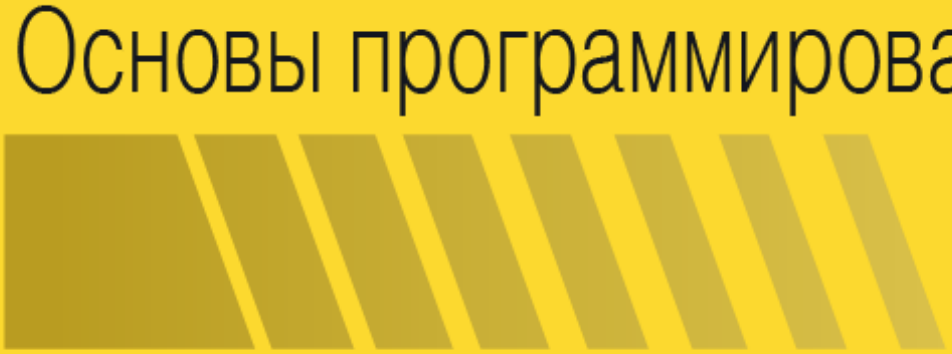


COURSE

Основы программирования



JS

Объектная модель документа (DOM)

shesnokov@gmail.com

ОГЛАВЛЕНИЕ

План лекции 3

Теоретическая часть 3

Объект document. Поиск элементов. 4

Свойства объекта document 7

Объект Node. Навигация по DOM 8

Создание, добавление и удаление элементов веб-станицы 11

Объект Element. Управление элементами 14

Изменение стиля элементов 18

Литература и другие источники 21

Вопросы для самопроверки 21

ПЛАН ЛЕКЦИИ

1. Объект Document. Поиск элементов;
2. Свойства объекта Document;
3. Объект Node. Навигация по DOM;
4. Создание, добавление и удаление элементов веб-страницы;
5. Объект Element. Управление элементами;
6. Изменение стиля элементов.

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

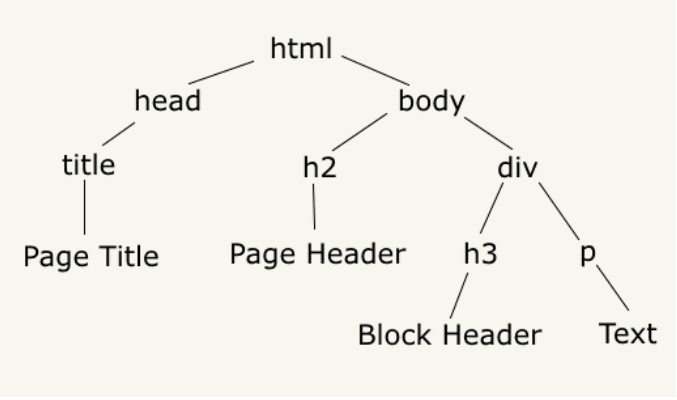
Одой из ключевых задач JavaScript является взаимодействие с пользователем и манипуляция элементами веб-страницы. Для JavaScript веб-страница доступна в виде объектной модели документа (document object model) или сокращенно DOM. DOM описывает структуру веб-станицы в виде древовидного представления и предоставляет разработчикам способ получить доступ к отдельным элементам веб-станицы.

Важно не путать понятия BOM (Browser Object Model - объектная модель браузера) и DOM (объектная модель документа). Если BOM предоставляет доступ к браузеру и его свойствам в целом, то DOM предоставляет доступ к отдельной веб-странице или html-документу и его элементам.

Например, рассмотрим простейшую страницу:

```
<!DOCTYPE html>
<html>
<head>
    <title>Page Title</title>
</head>
<body>
    <h2>Page Header</h2>
    <div>
        <h3>Block Header</h3>
        <p>Text</p>
    </div>
</body>
</html>
```

Дерево DOM для этой страницы будет выглядеть следующим образом:



Таким образом, все компоненты упорядочены в DOM иерархическим образом, где каждый компонент представляет отдельный узел. То есть каждый элемент, например, элемент div, представляет собой узел. Но также и текст внутри элемента представляет отдельный узел.

Существует следующие виды узлов:

- **Element**: html-элемент.
- **Attr**: атрибут html-элемента.
- **Document**: корневой узел html-документа.
- **DocumentType**: DTD или тип схемы XML-документа.
- **DocumentFragment**: место для временного хранения частей документа.
- **EntityReference**: ссылка на сущность XML-документа.
- **ProcessingInstruction**: инструкция обработки веб-страницы.
- **Comment**: элемент комментария.
- **Text**: текст элемента.
- **CDATASection**: секция CDATA в документе XML.
- **Entity**: необработанная сущность DTD.
- **Notation**: нотация, объявленная в DTD.

Несмотря на такое большое количество типов узлов, как правило, мы будем работать только с некоторыми из них.

ОБЪЕКТ DOCUMENT. ПОИСК ЭЛЕМЕНТОВ.

Для работы со структурой DOM в JavaScript предназначен объект document, который определен в глобальном объекте window. Объект document предоставляет ряд свойств и методов для управления элементами страницы.

Для поиска элементов на странице применяются следующие методы:

- **getElementById(value)**: выбирает элемент, у которого атрибут id равен value.
- **getElementsByTagName(value)**: выбирает все элементы, у которых тег равен value.
- **getElementsByClassName(value)**: выбирает все элементы, которые имеют класс value.
- **querySelector(value)**: выбирает первый элемент, который соответствует css-селектору value.
- **querySelectorAll(value)**: выбирает все элементы, которые соответствуют css-селектору value.

Например, найдем элемент по id:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
</head>
<body>
    <div>
        <h3 id="header">Block Header</h3>
        <p>Text</p>
    </div>
<script>
var headerElement = document.getElementById("header");
document.write("Текст заголовка: " + headerElement.innerText);
</script>
</body>
```

```
</html>
```

С помощью вызова document.getElementById("header") находим элемент, у которого id="header". А с помощью свойства innerText можно получить текст найденного элемента.

Поиск по определенному тегу:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
</head>
<body>
    <div>
        <h3>Заголовок</h3>
        <p>Первый абзац</p>
        <p>Второй абзац</p>
    </div>
<script>
var pElements = document.getElementsByTagName("p");

for (var i = 0; i < pElements.length; i++) {
    document.write("Текст параграфа: " + pElements[i].innerText + "<br/>");
}
</script>
</body>
</html>
```

С помощью вызова document.getElementsByTagName("p") находим все элементы параграфов. Этот вызов возвращает массив найденных элементов. Поэтому, чтобы получить отдельные элементы массива, необходимо пробежаться по ним в цикле.

Если нам надо получить только первый элемент, то можно к первому элементу найденной коллекции объектов:

```
var pElement = document.getElementsByTagName("p")[0];
document.write("Текст параграфа: " + pElement.innerText);
```

Получение элемента по классу:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
</head>
<body>
    <div class="article">
        <h3>Заголовок статьи</h3>
        <p class="text">Первый абзац</p>
        <p class="text">Второй абзац</p>
    </div>
<script>
var articleDiv = document.getElementsByClassName("article")[0];
    console.log(articleDiv);
```

```
var textElems = document.getElementsByClassName("text");
for (var i = 0; i < textElems.length; i++) {
    console.log(textElems[i]);
}
</script>
</body>
</html>
```

Выбор по селектору css:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
</head>
<body>
    <div class="annotation">
        <p>Аннотация статьи</p>
    </div>
    <div class="text">
        <p>Первый абзац</p>
        <p>Второй абзац</p>
    </div>
<script>
var elem = document.querySelector(".annotation p");
document.write("Текст селектора: " + elem.innerText);
</script>
</body>
</html>
```

Выражение document.querySelector(".annotation p") находит элемент, который соответствует селектору .annotation p. Если на странице несколько элементов, соответствующих селектору, то метод выберет первый из них.

Чтобы получить все элементы по селектору, можно подобным образом использовать метод document.querySelectorAll, который возвращает массив найденных элементов:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
</head>
<body>
    <div class="annotation">
        <p>Аннотация статьи</p>
    </div>
    <div class="text">
        <p>Первый абзац</p>
        <p>Второй абзац</p>
    </div>
<script>
var elems = document.querySelectorAll(".text p");
```

```
for (var i = 0; i < elems.length; i++) {
    document.write("Текст селектора " + i + ": " + elems[i].innerText + "<br/>");
}
</script>
</body>
</html>
```

СВОЙСТВА ОБЪЕКТА DOCUMENT

Кроме ранее рассмотренных методов объект document позволяет обратиться к определенным элементам веб-страницы через свойства:

- **documentElement**: предоставляет доступ к корневому элементу <html>.
- **body**: предоставляет доступ к элементу <body> на веб-странице.
- **images**: содержит коллекцию всех объектов изображений (элементов img).
- **links**: содержит коллекцию ссылок - элементов <a> и <area>, у которых определен атрибут href.
- **anchors**: предоставляет доступ к коллекции элементов <a>, у которых определен атрибут name.
- **forms**: содержит коллекцию всех форм на веб-странице.

Эти свойства не предоставляют доступ ко всем элементам, однако позволяют получить наиболее часто используемые элементы на веб-странице. Например, получим корневой узел документа:

```
var container = document.documentElement;
```

Получим все изображения на странице:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
</head>
<body>
    <img src="picure1.png" alt="Картинка 1" />
    <img src="picure2.png" alt="Картинка 2" />
    <img src="picure3.png" alt="Картинка 3" />
<script>
var images = document.images;
// изменим первое изображение
images[0].src="pics/picture_4.jpg";
images[0].alt="Новая картинка";
// перебирем все изображения
for(var i=0; i<images.length;i++){

    document.write("<br/>" + images[i].src);
    document.write("<br/>" + images[i].alt);
}
</script>
</body>
</html>
```

Подобно тому, как в коде html мы можем установить атрибуты у элемента img, так и в коде javascript мы можем через свойства src и alt получить и установить значения этих атрибутов.

Рассмотрим получение всех ссылок на странице:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
</head>
<body>
    <a href="article1.html">Статья 1</a>
    <a href="article2.html">Статья 2</a>
    <a href="article3.html">Статья 3</a>
<script>
var links = document.links;

for(var i=0; i<links.length;i++){

    document.write("<br/>" + links[i].innerText);
    document.write("<br/>" + links[i].href);
}
</script>
</body>
</html>
```

Опять же, так как у ссылки определен атрибут href, то при переборе ссылок мы можем получить его значение.

ОБЪЕКТ NODE. НАВИГАЦИЯ ПО DOM

Каждый отдельный узел, будь то html-элемент, его атрибут или текст, в структуре DOM представлен объектом Node. Этот объект предоставляет ряд свойств, с помощью которых мы можем получить информацию о данном узле:

- **childNodes**: содержит коллекцию дочерних узлов.
- **firstChild**: возвращает первый дочерний узел текущего узла.
- **lastChild**: возвращает последний дочерний узел текущего узла.
- **previousSibling**: возвращает предыдущий элемент, который находится на одном уровне с текущим.
- **nextSibling**: возвращает следующий элемент, который находится на одном уровне с текущим.
- **ownerDocument**: возвращает корневой узел документа.
- **parentNode**: возвращает элемент, который содержит текущий узел.
- **nodeName**: возвращает имя узла.
- **nodeType**: возвращает тип узла в виде числа.
- **nodeValue**: возвращает или устанавливает значение узла в виде простого текста.

Переберём все дочерние узлы элемента:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
```

```
</head>
<body>
    <div class="article">
        <h3>Заголовок статьи</h3>
        <p>Первый абзац</p>
        <p>Второй абзац</p>
    </div>
<script>
var articleDiv = document.querySelector("div.article");
var nodes = articleDiv.childNodes;
for(var i=0; i<nodes.length;i++){

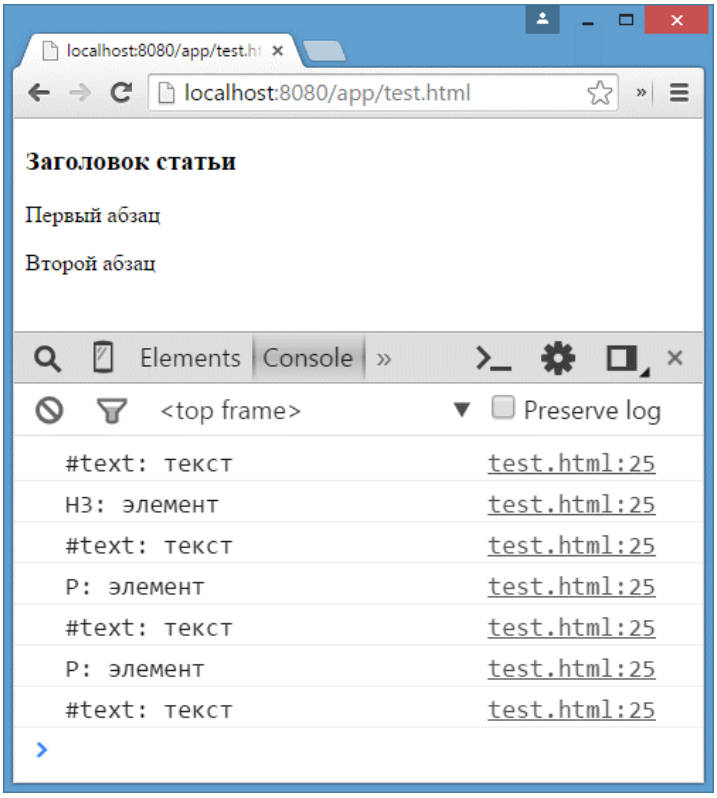
    var type = "";
    if(nodes[i].nodeType===1)
        type="элемент";
    else if(nodes[i].nodeType===2)
        type="атрибут";
    else if(nodes[i].nodeType===3)
        type="текст";

    console.log(nodes[i].nodeName + ": " + type);
}
</script>
</body>
</html>
```

С помощью метода document.querySelector("div.article") выбираем элемент div с классом article и пробегаемся по его дочерним узлам. И в цикле выводим имя узла и его тип с помощью свойств nodeName и nodeType. Каждому типу соответствует определенное число:

nodeType	Тип узла
1	элемент
2	атрибут
3	текст

Но несмотря на то, что в блоке div на странице только три узла: h3 и 2 параграфа, но консоль отобразит нам пять узлов.



Дело в том, что пробелы между узлами также считаются за отдельные текстовые узлы. Если бы пробелов не было:

```
div class="article"><h3>Заголовок статьи</h3><p>Первый абзац</p><p>Второй абзац</p></div>
```

То при переборе мы бы обнаружили только три дочерних узла, как и ожидалось.

Используя свойства nextSibling и previousSibling можно также пройтись по узлам в прямом или обратном порядке. Например, для прохода по элементам на той же станице мы могли бы использовать следующий код javascript:

```
var articleDiv = document.querySelector("div.article");

// получаем первый дочерний элемент
var node = articleDiv.firstChild;
console.log(node.nodeName);
// обращаемся к следующему узлу, пока он определен
while((node=node.nextSibling)!==null){

    console.log(node.nodeName);
}
```

Тут все аналогично, только в цикле мы обращаемся к следующему узлу, который находится на одном уровне с текущим (или сестринскому узлу). И результат будет такой же.

Также можно перебрать узлы в обратном порядке: сначала получаем последний узел, а затем обращаемся к предыдущему сестринскому узлу:

```
var articleDiv = document.querySelector("div.article");
```

```
// получаем первый дочерний элемент
var node = articleDiv.lastChild;
console.log(node.nodeName);
// обращаемся к предыдущему узлу, пока он определен
while((node=node.previousSibling)!==null){
    console.log(node.nodeName);
}
```

СОЗДАНИЕ, ДОБАВЛЕНИЕ И УДАЛЕНИЕ ЭЛЕМЕНТОВ ВЕБ-СТАНИЦЫ

Для создания элементов объект document имеет следующие методы:

- **createElement(elementName)**: создает элемент html, тег которого передается в качестве параметра. Возвращает созданный элемент
- **createTextNode(text)**: создает и возвращает текстовый узел. В качестве параметра передается текст узла.

```
var elem = document.createElement("div");
var elemText = document.createTextNode("Привет мир");
```

Таким образом, переменная elem будет хранить ссылку на элемент div. Однако одного создания элементов недостаточно, их еще надо добавить на веб-страницу.

Для добавления элементов мы можем использовать один из методов объекта Node:

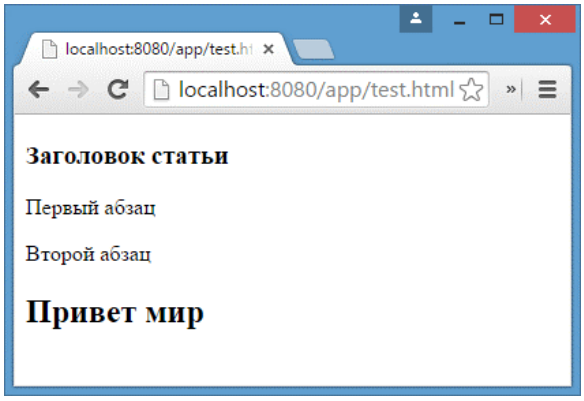
- **appendChild(newNode)**: добавляет новый узел newNode в конец коллекции дочерних узлов.
- **insertBefore(newNode, referenceNode)**: добавляет новый узел newNode перед узлом referenceNode.

Используем метод appendChild:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
</head>
<body>
    <div class="article">
        <h3>Заголовок статьи</h3>
        <p>Первый абзац</p>
        <p>Второй абзац</p>
    </div>
<script>
var articleDiv = document.querySelector("div.article");
// создаем элемент
var elem = document.createElement("h2");
// создаем для него текст
var elemText = document.createTextNode("Привет мир");
// добавляем текст в элемент в качестве дочернего элемента
elem.appendChild(elemText);
// добавляем элемент в блок div
articleDiv.appendChild(elem);
</script>
```

```
</body>
</html>
```

Сначала создаем обычный элемент заголовка h2 и текстовый узел. Затем текстовый узел добавляем в элемент заголовка. Затем заголовок добавляем в один из элементов веб-страницы:



Однако нам необязательно для определения текста внутри элемента создавать дополнительный текстовый узел, так как мы можем воспользоваться свойством textContent и напрямую ему присвоить текст:

```
var elem = document.createElement("h2");
elem.textContent = "Привет мир";
```

В этом случае текстовый узел будет создан неявно при установке текста.

Теперь рассмотрим, как аналогичный элемент добавить в начало коллекции дочерних узлов блока div:

```
var articleDiv = document.querySelector("div.article");
// создаем элемент
var elem = document.createElement("h2");
// создаем для него текст
var elemText = document.createTextNode("Привет мир");
// добавляем текст в элемент в качестве дочернего элемента
elem.appendChild(elemText);
// получаем первый элемент, перед которым будет идти добавление
var firstElem = articleDiv.firstChild.nextSibling;
// добавляем элемент в блок div перед первым узлом
articleDiv.insertBefore(elem, firstElem);
```

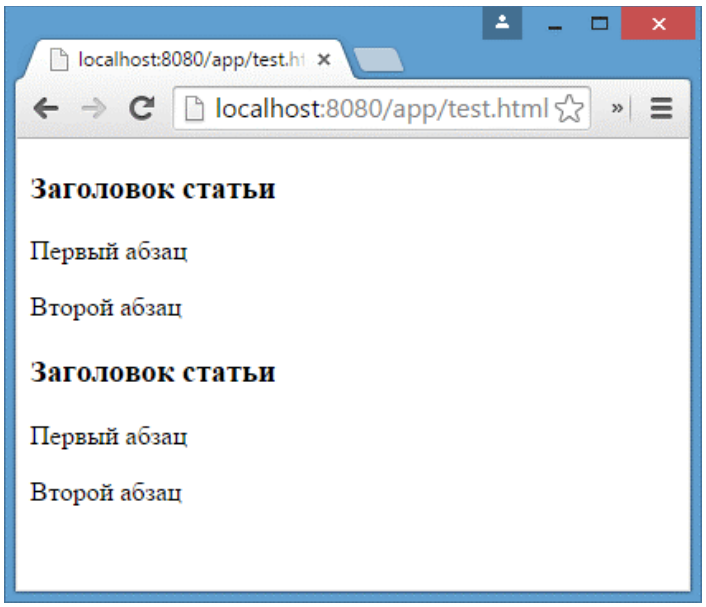
Если нам надо вставить новый узел на второе, третье или любое другое место, то нам надо найти узел, перед которым надо вставлять, с помощью комбинаций свойств firstChild/lastChild и nextSibling/previousSibling.

Иногда элементы бывают довольно сложными по составу, и гораздо проще их скопировать, чем с помощью отдельных вызовов создавать из содержимое. Для копирования уже имеющихся узлов у объекта Node можно использовать метод cloneNode():

```
var articleDiv = document.querySelector("div.article");
// клонируем элемент articleDiv
var newArticleDiv = articleDiv.cloneNode(true);
// добавляем в конец элемента body
```

```
document.body.appendChild(newArticleDiv);
```

В метод cloneNode() в качестве параметра передается логическое значение: если передается true, то элемент будет копироваться со всеми дочерними узлами; если передается false - то копируется без дочерних узлов. То есть в данном случае мы копируем узел со всем его содержимым и потом добавляем в конец элемента body.



Для удаления элемента вызывается метод removeChild() объекта Node. Этот метод удаляет один из дочерних узлов:

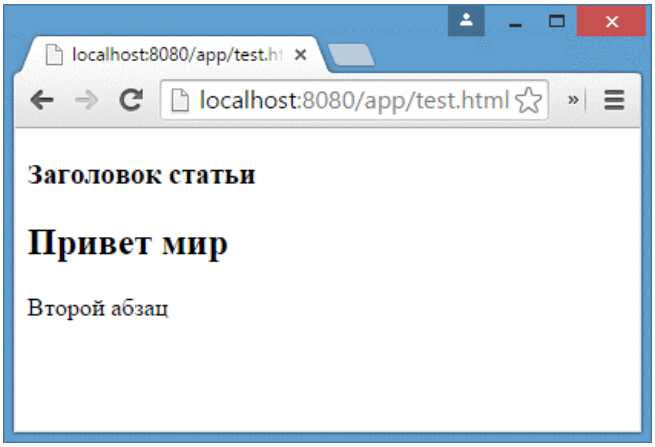
```
var articleDiv = document.querySelector("div.article");
// находим узел, который будем удалять - первый параграф
var removableNode = document.querySelectorAll("div.article p")[0];
// удаляем узел
articleDiv.removeChild(removableNode);
```

В данном случае удаляется первый параграф из блока div.

Для замены элемента применяется метод replaceChild(newNode, oldNode) объекта Node. Этот метод в качестве первого параметра принимает новый элемент, который заменяет старый элемент oldNode, передаваемый в качестве второго параметра.

```
var articleDiv = document.querySelector("div.article");
// находим узел, который будем заменять - первый параграф
var oldNode = document.querySelectorAll("div.article p")[0];
// создаем элемент
var newNode = document.createElement("h2");
// создаем для него текст
var elemText = document.createTextNode("Привет мир");
// добавляем текст в элемент в качестве дочернего элемента
newNode.appendChild(elemText);
// заменяем старый узел новым
articleDiv.replaceChild(newNode, oldNode);
```

В данном случае заменяем первый параграф заголовком h2:



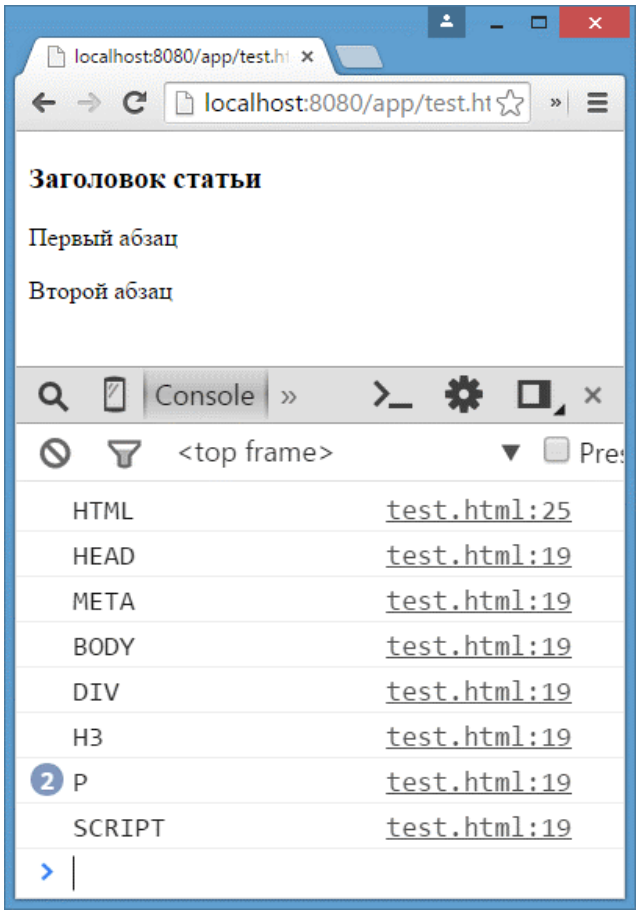
ОБЪЕКТ ELEMENT. УПРАВЛЕНИЕ ЭЛЕМЕНТАМИ

Кроме методов и свойств объекта Node в JavaScript мы можем использовать свойства и методы объектов Element. Важно не путать эти два объекта: Node и Element. Node представляет все узлы веб-станицы, в то время как объект Element представляет непосредственно только html-элементы. То есть объекты Element - это фактически те же самые узлы - объекты Node, у которых тип узла (свойство nodeType) равно 1.

Одним из ключевых свойств объекта Element является свойство tagName, которое возвращает тег элемента. Например, получим все элементы, которые есть на странице:

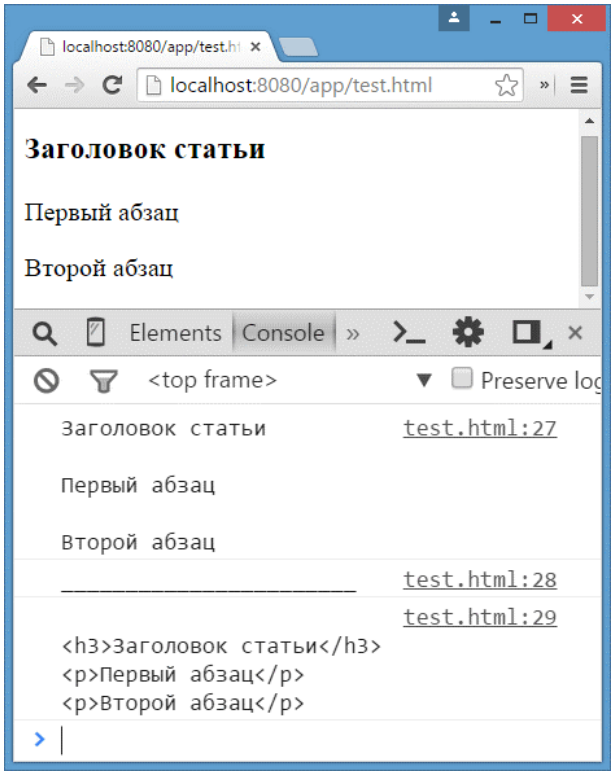
```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
</head>
<body>
<div class="article">
<h3>Заголовок статьи</h3>
<p>Первый абзац</p>
<p>Второй абзац</p>
</div>
<script>
function getChildren(elem){
    for(var i in elem.childNodes){
        if(elem.childNodes[i].nodeType===1){
            console.log(elem.childNodes[i].tagName);
            getChildren(elem.childNodes[i]);
        }
    }
}
var root = document.documentElement;
console.log(root.tagName);
getChildren(root);
</script>
</body>
</html>
```

Здесь вначале получаем те корневого элемента <html> и затем с помощью рекурсивной функции getChildren получаем все вложенные элементы:



Для получения или установки текстового содержимого элемента мы можем использовать свойство innerText, а для получения или установки кода html - свойство innerHTML:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
</head>
<body>
<div class="article">
<h3>Заголовок статьи</h3>
<p>Первый абзац</p>
<p>Второй абзац</p>
</div>
<script>
var articleDiv = document.querySelector("div.article");
console.log(articleDiv.innerText);
console.log("_____");
console.log(articleDiv.innerHTML);
</script>
</body>
</html>
```



Надо отметить, что свойство innerText во многом аналогично свойству textContent. То есть следующие вызовы будут равноценны:

```
var pElement = document.querySelectorAll("div.article p")[0];
pElement.innerText = "hello";
pElement.textContent = "hello";
```

Установка кода html у элемента:

```
var articleDiv = document.querySelector("div.article");
articleDiv.innerHTML ="<h2>Hello World!!!</h2><p>bla bla bla</p>";
```

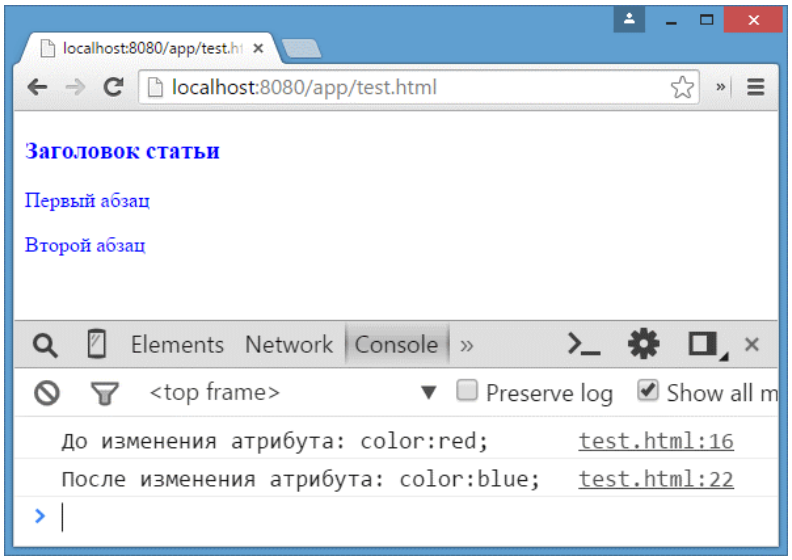
Среди методов объекта Element можно отметить методы управления атрибутами:

- **getAttribute(attr)**: возвращает значение атрибута attr.
- **setAttribute(attr, value)**: устанавливает для атрибута attr значение value. Если атрибута нет, то он добавляется.
- **removeAttribute(attr)**: удаляет атрибут attr и его значение.

Работа с атрибутами:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
</head>
<body>
<div class="article" style="color:red;">
<h3>Заголовок статьи</h3>
<p>Первый абзац</p>
<p>Второй абзац</p>
```

```
</div>
<script>
var articleDiv = document.querySelector("div.article");
// получаем атрибут style
var styleValue = articleDiv.getAttribute("style");
console.log("До изменения атрибута: " + styleValue);
// удаляем атрибут
articleDiv.removeAttribute("style");
// добавляем заново атрибут style
articleDiv.setAttribute("style", "color:blue;");
styleValue = articleDiv.getAttribute("style");
console.log("После изменения атрибута: " + styleValue);
</script>
</body>
</html>
```



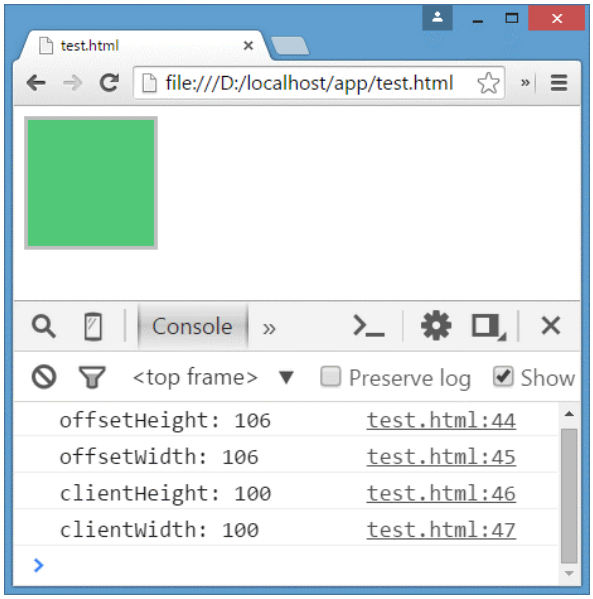
Элементы имеют ряд свойств, которые позволяют определить размер элемента. Но важно понимать разницу между всеми этими свойствами.

Свойства ***offsetWidth*** и ***offsetHeight*** определяют соответственно ширину и высоту элемента в пикселях. В ширину и высоту включается граница элемента.

Свойства ***clientWidth*** и ***clientHeight*** также определяют ширину и высоту элемента в пикселях, но уже без учета границы:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <style>
    #rect {
        width: 100px;
        height: 100px;
        background: #50c878;
        border: 3px solid silver;
    }
```

```
    </style>
</head>
<body>
<div id="rect"></div>
<script>
var rect = document.getElementById("rect");
console.log("offsetHeight: " + rect.offsetHeight);
console.log("offsetWidth: " + rect.offsetWidth);
console.log("clientHeight: " + rect.clientHeight);
console.log("clientWidth: " + rect.clientWidth);
</script>
</body>
</html>
```



Поскольку у блока div определена граница в 3 пикселя, то по сравнению с clientHeight/clientWidth к offsetHeight/offsetWidth добавляет по 6 пикселей.

Для определения позиции элемента наиболее эффективным способом является метод getBoundingClientRect().

Этот метод возвращает объект со свойствами top, bottom, left, right, которые указывают на смещение элемента относительно верхнего левого угла браузера:

```
var rect = document.getElementById("rect");
var clientRect  = rect.getBoundingClientRect();
console.log("top: " + clientRect.top);
console.log("bottom: " + clientRect.bottom);
console.log("left: " + clientRect.left);
console.log("right: " + clientRect.right);
```

ИЗМЕНЕНИЕ СТИЛЯ ЭЛЕМЕНТОВ

Для работы со стилевыми свойствами элементов в JavaScript применяются, главным образом, два подхода:

- Изменение свойства style.

- Изменение значения атрибута class.

Свойство style представляет сложный объект для управления стилем и напрямую сопоставляется с атрибутом style html-элемента. Этот объект содержит набор свойств CSS: element.style.свойствоCSS. Например, установим цвет шрифта:

```
var root = document.documentElement;
// устанавливаем стиль
root.style.color = "blue";
// получаем значение стиля
document.write(root.style.color); // blue
```

В данном случае название свойства color совпадает со свойством css. Аналогично мы могли бы установить цвет с помощью css:

```
html{
    color:blue;
}
```

Однако ряд свойств css в названиях имеют дефис, например, font-family. В JavaScript для этих свойств дефис не употребляется. Только первая буква, которая идет после дефиса, переводится в верхний регистр:

```
var root = document.documentElement;
root.style.fontFamily = "Verdana";
```

С помощью свойства className можно установить атрибут class элемента html. Например:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <style>
    .blueStyle{
        color:blue;
        font-family:Verdana;
    }
    .article{
        font-size:20px;
    }
    </style>
</head>
<body>
<div class="article">
<h3>Заголовок статьи</h3>
<p>Первый абзац</p>
<p>Второй абзац</p>
</div>
<script>
var articleDiv = document.querySelector("div.article");
// установка нового класса
articleDiv.className = "blueStyle";
// получаем название класса
document.write(articleDiv.className);
```

```
</script>
</body>
</html>
```

Благодаря использованию классов не придется настраивать каждое отдельное свойство css с помощью свойства style.

Но при этом надо учитывать, что прежнее значение атрибута class удаляется. Поэтому, если нам надо добавить класс, надо объединить его название со старым классом:

```
articleDiv.className = articleDiv.className + " blueStyle";
```

И если надо вовсе удалить все классы, то можно присвоить свойству пустую строку:

```
articleDiv.className = "";
```

Выше было рассмотрено, как добавлять классы к элементу, однако для управления множеством классов гораздо удобнее использовать свойство classList. Это свойство представляет объект, реализующий следующие методы:

- **add(className)**: добавляет класс className.
- **remove(className)**: удаляет класс className.
- **toggle(className)**: переключает у элемента класс на className. Если класса нет, то он добавляется, если есть, то удаляется.

Например:

```
var articleDiv = document.querySelector("div.article");
// удаляем класс
articleDiv.classList.remove("article");
// добавляем класс
articleDiv.classList.add("blueStyle");
// переключаем класс
articleDiv.classList.toggle("article");
```

ЛИТЕРАТУРА И ДРУГИЕ ИСТОЧНИКИ

1. Стандарт HTML https://html.spec.whatwg.org/multipage/window-object.html#the-window-object
2. Дакетт Джон. HTML и CSS. Разработка и дизайн веб-сайтов. Пер. с англ. – Эксмо, 2017. – 480 с.
3. Прасти Н. Введение в ECMAScript 6. Изд. ДМК, 2016. – 176 с.

ВОПРОСЫ ДЛЯ САМОПРОВЕРКИ

1. Дайте определение объектной модели документа (DOM).
2. Какие элементы составляют основу DOM?
3. Какие варианты поиска элементов на странице предоставляет DOM?
4. Что такое узел (node) DOM-структуры?